

LICENSING OFFICER

Review queue

LICENSING OFFICER

Review queue

3 need review · 0 waiting on operators · 4 decided

Refreshed just now · updates every 30 s



PermitFlow

PermitFlow: how it is built

Scope, architecture, the decisions that make the brief's guarantees true, the pipeline, the evidence, and the gaps. Every claim names its file.

mitFlow

Needs review 3

Waiting on operator 0

Decided 4

All 7

Q Search the queue

REFERENCE

BUSINESS · APPLICANT

STATUS · CHECKS

PF-2026-001009

Revision 1 · 2 open

Kopi & Kaya Toast House Pte. Ltd.

Tan Wei Ling · 10 Jalan Besar #01-12 · just now

Continue review →

Under Review

2 to check

just now

Submitted 19 Sep 2026

PF-2026-001001

Revision 1

Scenario Kopi House Pte. Ltd.

Tan Wei Ling · 10 Jalan Besar #01-12 · 14 h ago

Continue review →

Under Review

4 to check

14 h ago

Submitted 19 Sep 2026

PF-2026-001000

Revision 1

Scenario Kopi House Pte. Ltd.

Tan Wei Ling · 10 Jalan Besar #01-12 · 19 h ago

Continue review →

Under Review

4 to check

19 h ago

Submitted 18 Sep 2026

Flow: how it is built

re, the decisions that make the brief's guarantees true, the pipeline, the

e gaps. Every claim names its file.

Three use cases, one status table, five things being evaluated.

UC1 **Operator submission and resubmission**

● Built

Form with sections, drag-and-drop uploads, live AI status per document, progress, feedback on top, edit only flagged parts, unlimited rounds, history.

UC2 **Officer review and feedback**

● Built

Full submission, AI results, contextual feedback with templates, status change notifies, changed sections highlighted, compare revisions, resolution tracked, audit trail.

UC3 **On-site assessment and post-site clarification**

● Deferred

The three post-site statuses and their transitions exist and are tested; the checklist model and screens are not built. Recorded in SCOPE.md with what production needs.

STATUS TABLE

Status table: 12 internal statuses with a different label per role.
Implemented as data in domain/labels.py, verbatim from the brief, plus draft and withdrawn as written assumptions.

WHAT IS EVALUATED, AND WHERE IT IS IN THIS DECK

Scope judgement	slides 3 and 4
Production readiness	slides 10 to 12
AI tool usage	slide 13
Code quality	slides 5 to 9
Documentation	docs/README.md is the index; docs/reviews/ASSESSMENT_TRACEABILITY.md maps every brief line to code, test and evidence

One polished, tested, deployed vertical slice. Everything else written down with what production would need.

● MUST**20 of 20 built**

- Form, uploads with allowlist, magic bytes and 10 MB
- AI check per document with live status
- Explicit state machine, 14 statuses, role labels
- Feedback tied to a section or document, templates
- Targeted resubmission, immutable revisions, compare
- Audit trail, in-app notifications
- Server-side authorization, error envelope
- Tests per layer, CI, Railway, README, SCOPE.md

● SHOULD**6 of 7 done**

- Queue groups and search
- Re-run a check (operator and officer)
- Delete a document while in draft
- Compare any two revisions
- Evaluation set: 14 cases, mock gate, live nightly
- Request logging with request id
- S7 admin persona: deferred to v0.4.0

● COULD**5 of 8 done**

- Dashboard summary
- Withdraw with a reason (new terminal status)
- Licence certificate on approval, preview, PDF
- Owner's domain, two environments, TLS
- Legal, privacy and accessibility review
- Open: officer assignment, image OCR, bundle download

● DEFERRED**13 items, each with a reason**

- UC3 checklist (1.5 days estimated)
- Email delivery (mocked, in-app only)
- Worker queue for AI checks (in-process today)
- Object storage (volume today)
- Registration, reset, MFA, SSO
- Configurable forms, OCR, push, WAF, virus scan
- Session in sessionStorage, US-region providers, one replica: disclosed with severity

Where the brief was ambiguous, the assumption is written down and enforced by code.

01 **A draft state exists before Application Received**

Needed for save and return; never shown to officers; deletable outright until submitted (US-045). SCOPE.md assumption 2 and 14.

02 **Under Review starts on an explicit officer action**

So the actor lands in the audit trail. Start review is a transition row, not a side effect. Assumption 3.

03 **Requesting resubmission needs at least one open feedback item**

Guard on the transition; the button is disabled with the reason. Assumption 4, domain/workflow.py.

04 **Addressed is automatic, resolved is a decision**

The linked section or document changed in the next revision: addressed. The officer confirms: resolved. Assumption 7.

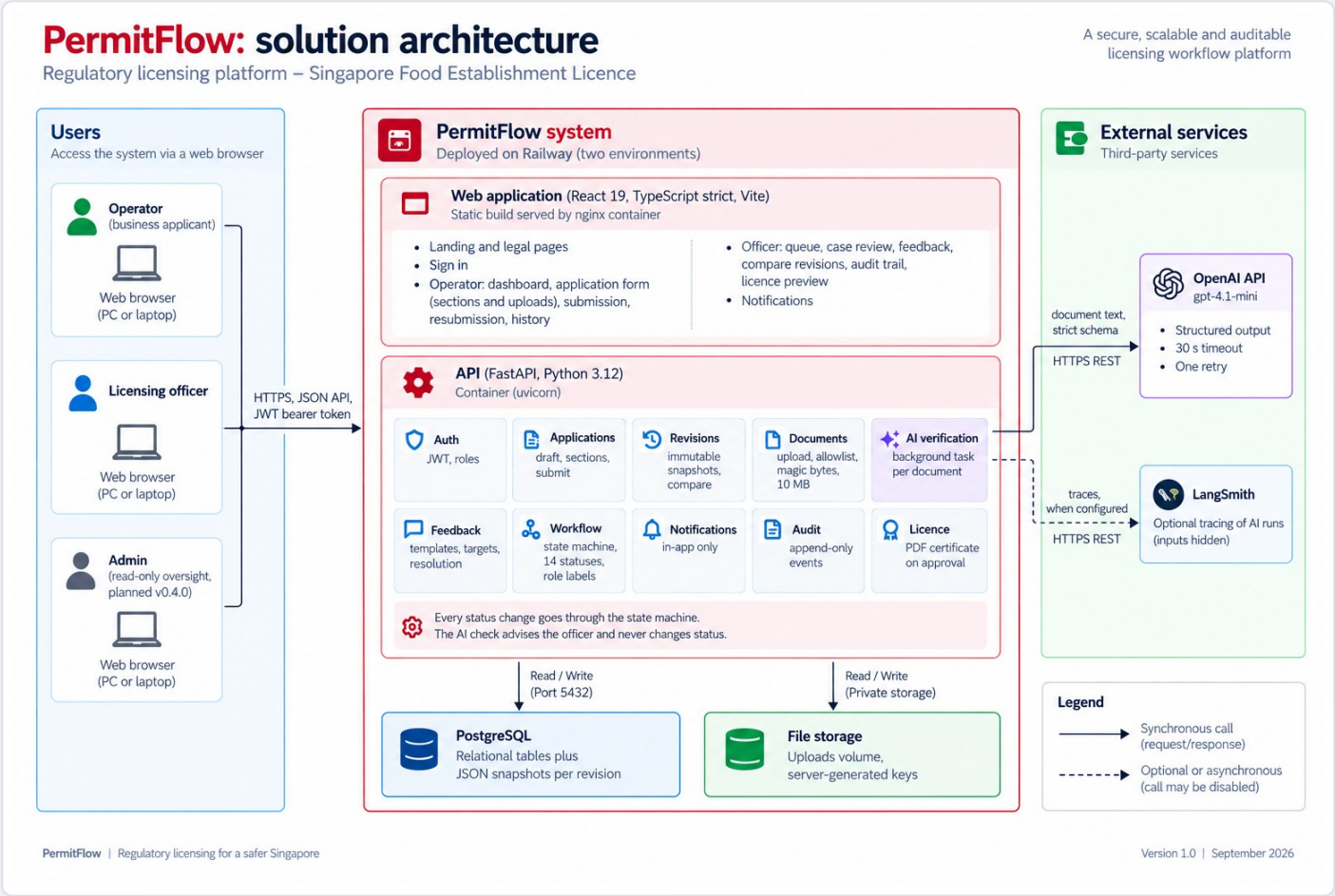
05 **Only flagged sections may change on resubmission: enforced**

The API returns 403 for any other change. A product decision open to reversal. Assumption 13, domain/resolution.py.

06 **Operators see Pending Approval as a waiting state**

The brief's own table gives that label; the officer-only label Route to Approval and the decision note never reach an operator. Assumption 17.

A modular monolith: one API in strict layers, one web app, one database, an isolated AI module.



Layers, enforced

api, services, domain, repositories, models. domain is pure Python: no SQLAlchemy, no FastAPI, no I/O. tests/unit/test_layering.py fails the build if a layer imports downward the wrong way. ADR-001.

PostgreSQL only

Relations for users, applications, feedback and audit; every submitted revision an immutable JSON snapshot. Row locks and a version column for concurrency. ADR-002.

AI in-process, behind an interface

Background task per document, result polled every 2 s, stale runs marked failed on restart. VerificationProvider with OpenAI and a deterministic mock. ADR-004, the worker is next.

Stack

FastAPI, SQLAlchemy 2, Pydantic v2, Alembic; React 19, TypeScript strict, Vite, TanStack Query, React Hook Form, Zod; pytest, vitest, Playwright. ADR-009.

The workflow is a table, not a set of if statements.

CHOSE

The fourteen statuses and every allowed move as data: from, to, actor, guard, label. One function, `transition()`, consults it. `available_actions()` gives the UI its buttons with the reason a button is disabled.

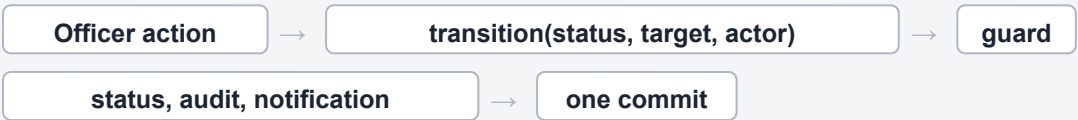
OVER

Status changes scattered through services, or a workflow library with its own model of the world.

BECAUSE

A table can be compared with the brief's status table line by line and tested exhaustively. Adding a status is a row. Reject is possible from every non-terminal post-submission state, so no case can get stuck.

THE PATH



EVIDENCE IN THE CODE

domain/workflow.py

18 transition rows plus withdraw from any post-submission state; guards: `is_complete`, `open_feedback_count`, `has_changes_to_flagged_targets`

domain/labels.py

the brief's table verbatim: internal status, officer label, operator label, badge tone

tests/unit/test_workflow.py

every (source, target, actor) combination: $14 \times 14 \times 3 = 588$ cases; terminal states have no exits; guards

POST /officer/applications/{id}/transition

row lock, `expected_version`, 409 `invalid_transition` with the allowed list, audit and notification in the same transaction

TRADE-OFF

Guards are Python callables, so a guard that needs data the context does not carry means extending the context. Accepted: the context has three fields and has not grown since Day 1.

Operators never see internal status codes, because the API cannot send them.

CHOSE

Role checked per router; ownership enforced in the one function that loads an application (another owner's case is a 404, not a 403); a separate response model per role, so the operator view has no field for the internal status or the officer label.

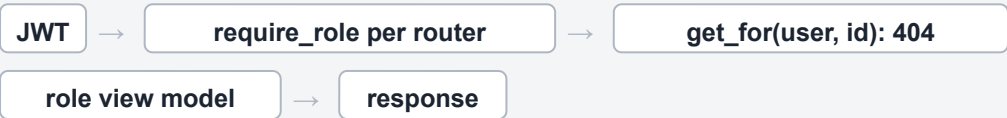
OVER

Checking roles in the UI, or one shared response shape with fields hidden client-side.

BECAUSE

The brief's constraint, operators cannot see the approval stage, becomes a property of the types and the tests, not a hope. A leaked field would be a schema change, which a reviewer sees in the diff.

THE PATH



EVIDENCE IN THE CODE

api/deps.py

get_current_user (JWT, active user), require_role('officer'), require_role('admin')

repositories/applications.py

get_for(user, id) applies the operator_id filter; NotFound for another owner

schemas/applications.py vs schemas/officer.py

operator view: operator label, released feedback without author, no internal status, no decision note before the outcome

tests/integration

an authorization test for every endpoint: wrong role 403, wrong owner 404, cross-application ids 404, admin read-only

TRADE-OFF

Two response models to keep in step when a field is added. Accepted: the layering test and the API table in ARCHITECTURE.md make a missed field visible.

The AI check advises. Strict schemas and deterministic rules decide what an answer is allowed to mean.

CHOSE

A provider interface with two implementations (OpenAI structured output, deterministic mock). A pinned wire schema, then a strict domain model (extra forbidden, confidence 0 to 1, status must agree with the issue list), then rules after the model: confidence under 0.6 or an injection phrase forces needs_review. The result is stored on the run and shown to the officer. It never touches status, feedback or editability.

OVER

Letting the model's answer drive the workflow, or free-text answers parsed by hand.

BECAUSE

The brief says AI assists and the officer decides. With strict schemas a bad answer becomes a flagged answer, never a state change. Over quota or on failure the run is stored unavailable and the application proceeds.

THE PATH



EVIDENCE IN THE CODE

`infra/ai/base.py`, `openai_provider.py`, `mock.py`

VerificationProvider; prompt_version 2026-09-19.3 stamped on every run; 30 s timeout, one retry

`domain/verification_rules.py`

VerificationResult (extra=forbid), apply_rules(threshold, injection phrases), find_injection_phrases

`services/verification.py`

the only caller of infra.ai; own session and transaction; text extraction capped (30 pages, 20 000 chars, 10 s); never raises out of the task

`evals/`: 14 golden cases, 2 injection cases, 21 fairness runs

mock gate on every push, live nightly, LangSmith experiment per run; docs/ai/AI_ASSURANCE.md

TRADE-OFF

Confidence is the model's own estimate and is uncalibrated; the injection heuristic is English-only phrases. Both recorded as next steps with the tools to fix them.

Nothing is lost between rounds: every submission is a frozen snapshot, and every change writes its audit row in the same transaction.

CHOSE

Each submit or resubmit stores Revision N: the form as JSON plus the document ids. The comparison between revisions is computed at read time from the snapshots. Audit events, notifications and the licence on approval are written in the same database transaction as the change; there is no update or delete path for audit rows.

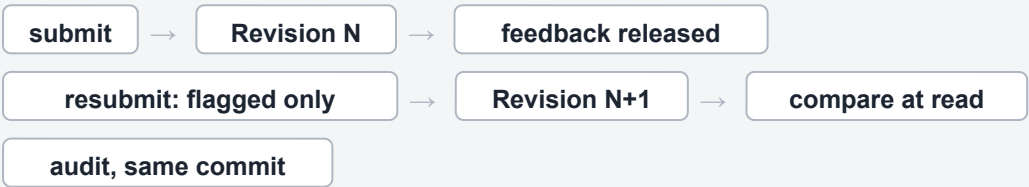
OVER

Editing one mutable record, storing diffs instead of snapshots, or a separate logging pipeline and database triggers.

BECAUSE

Never lost between rounds and complete audit trail become true by construction. The officer's compare view cannot disagree with what was submitted. A change without its audit row cannot exist, and vice versa.

THE PATH



EVIDENCE IN THE CODE

models: `application_revisions`, `audit_events`
immutable rows; raised_in_revision on feedback; sha256 on documents so an identical re-upload is no change

domain/diff.py, services/compare.py
field-level form diff and document add, remove, replace; GET /applications/{id}/compare?from=n&to=m

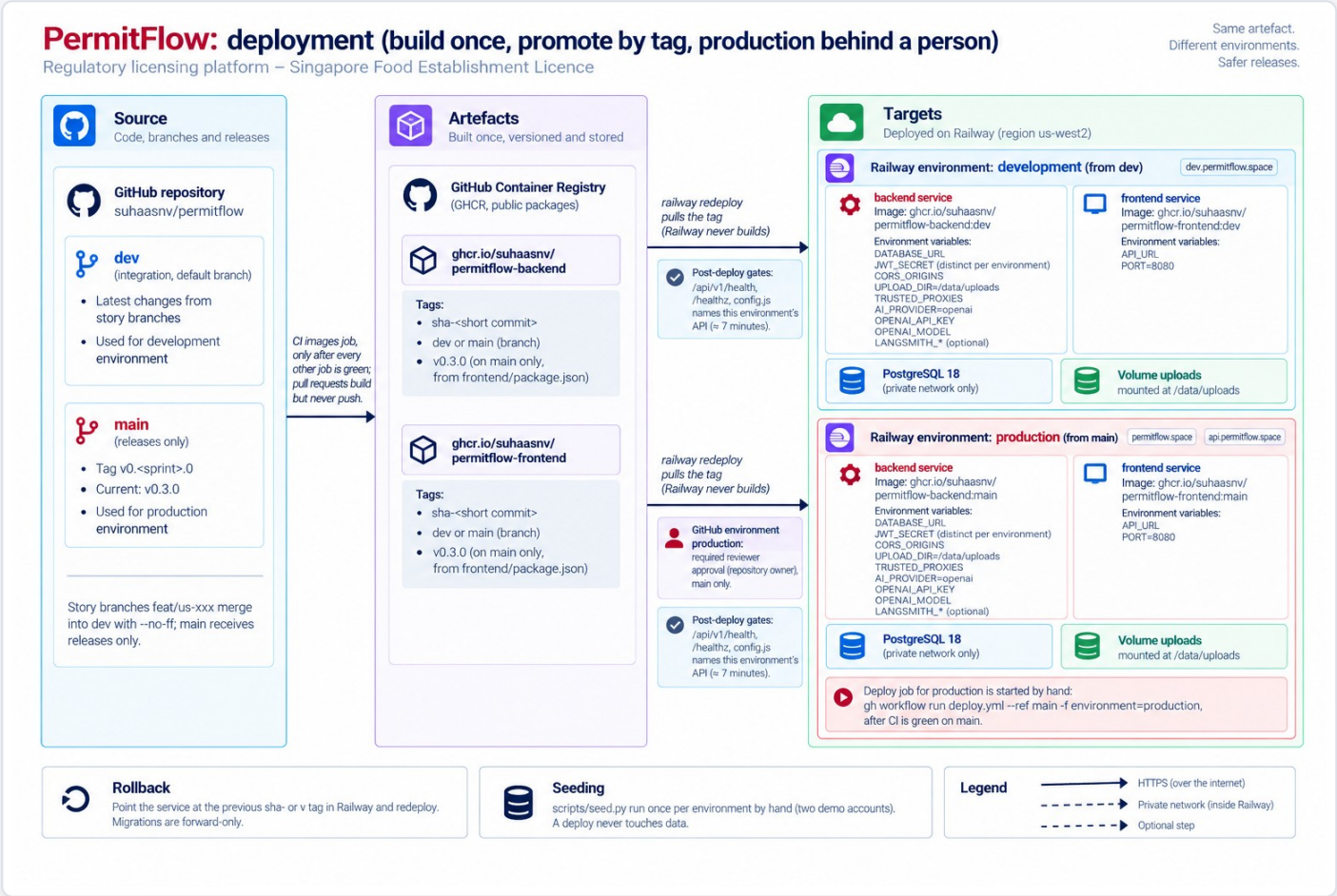
services/resubmission.py
SELECT FOR UPDATE, only flagged targets may change (403), at least one flagged target changed (422), Revision N+1, feedback to addressed, audit, notify, one commit

GET /officer/applications/{id}/audit
actor, role, plain-language summary from domain/audit_labels.py, payload, chronological; 748 backend tests include every event type

TRADE-OFF

Snapshots grow with rounds; at this form size a revision is a few kilobytes. A separate audit store would be the answer at a scale this product is not at.

Build once, promote by tag, production behind a person.



Images built once

CI builds both images after every other job is green and pushes them to GHCR with sha-<commit>, the branch, and v0.3.0 on main. Railway never builds; the tested artefact is the deployed artefact.

Two environments, nothing shared

development from dev, production from main. Own PostgreSQL 18, own uploads volume, own secrets, own tokens, own domains.

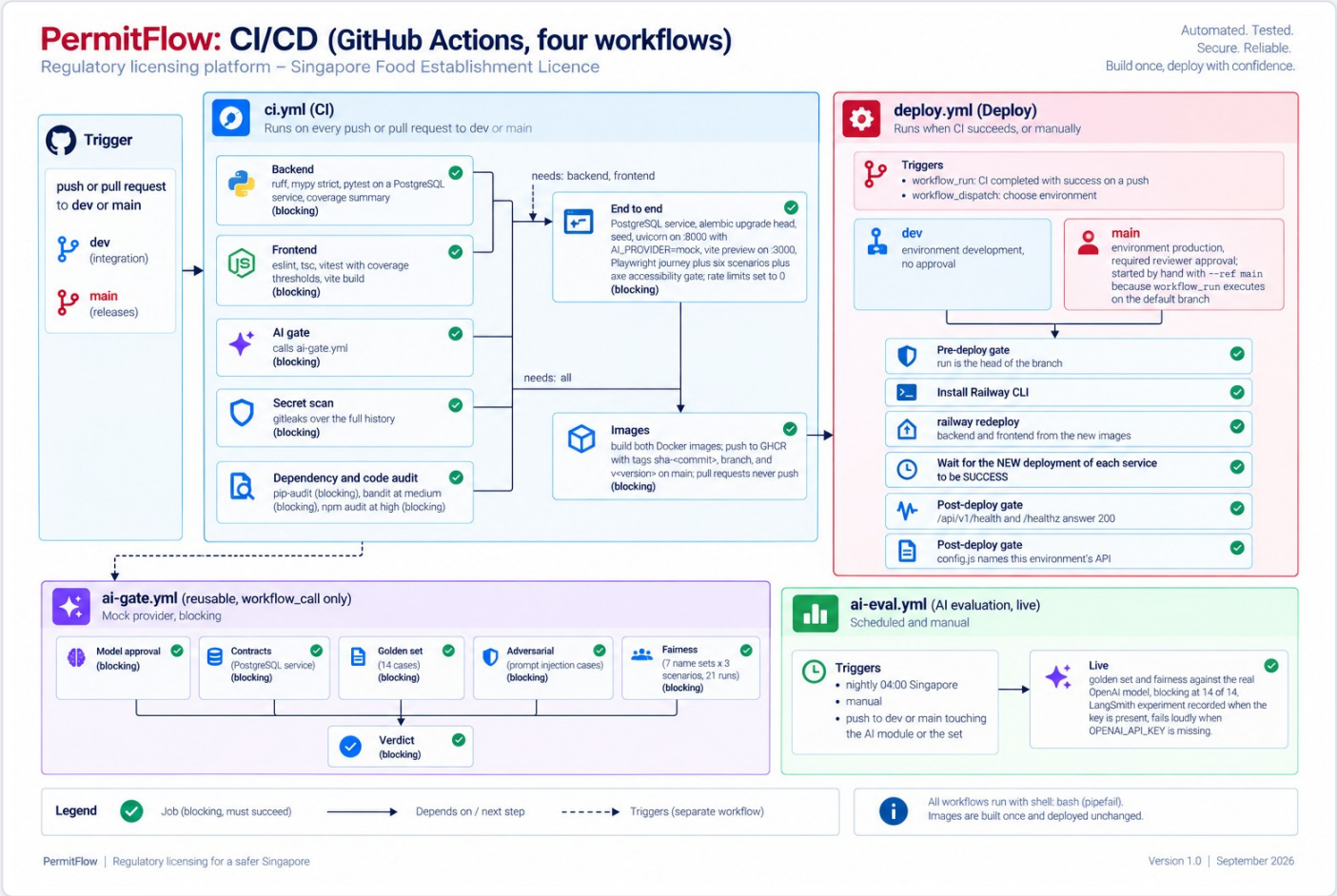
Gates

Production waits for the repository owner's approval in the GitHub environment. Both environments gate on /api/v1/health, /healthz and config.js naming the right API. Rollback is the previous tag.

Known limit

A workflow_run job executes on the default branch, dev, and the production environment admits only main, so production is deployed by hand: gh workflow run deploy.yml --ref main. Documented in OPERATIONS.md.

Seven CI jobs, an AI gate with six, a live evaluation, and a deploy that waits for health.



Everything blocks

Backend (ruff, mypy strict, 748 pytest on PostgreSQL), Frontend (eslint, tsc, 152 vitest with thresholds, build), gitleaks, pip-audit, bandit, npm audit, End to end (the whole stack in the runner, Playwright), Images. No job is advisory.

AI gate on every push

Model approval, contracts on a real database, 14 golden cases, adversarial injection cases, 21 fairness runs, verdict. On the mock provider so it is deterministic and free.

Live evaluation

Nightly and on AI-path pushes against the real OpenAI model: blocking at 14 of 14, each run recorded as a LangSmith experiment. Fails loudly when the key is missing.

Reproducible

The end-to-end job runs migrations, seed, uvicorn and vite preview inside the runner, with rate limits set to 0 and the mock provider. The same commands run locally.

.github/workflows: ci.yml, ai-gate.yml (workflow_call), ai-eval.yml, deploy.yml · shell: bash with pipefail everywhere

Checked by tests, gates and a written record.

748 backend tests, 96 % statements **152** frontend tests, 80.6 % **8** Playwright specs, axe over 23 states **23** threats, each with a control **20** readiness rows with severity

- ✓ **Input validation** Pydantic 422 with per-field details; upload allowlist, size streamed to 10 MB, magic bytes; zod schemas built from the server's form schema.
- ✓ **Error envelope** { error: { code, message, details } } on every route, including 401, 403, 422, 429 and 500 with a request id and no stack trace.
- ✓ **Concurrency** SELECT FOR UPDATE on the application; expected_version on transitions (409 version_conflict); stale AI runs recovered on restart.
- ✓ **Abuse limits** 240 requests and 20 sign-ins per client per minute with Retry-After; 20 open drafts; 60 checks per applicant and 1 000 per day, over quota stored unavailable.
- ✓ **Headers and secrets** CSP, HSTS, COOP, Permissions-Policy on both tiers; Argon2, JWT 8 h re-validated per request; gitleaks over the history; .env.example documents every variable.
- ✓ **Accessibility and legal** axe gate in CI over 23 screen states; labels never colour alone; policy pages and demonstration notices; fonts self-hosted.

Threat model

docs/security/THREAT_MODEL.md: T1 to T22 plus T1a, STRIDE per asset, the control for each, what is accepted for a demonstration and why.

Readiness review

docs/reviews/PRODUCTION_READINESS_REVIEW.md: 20 rows, severity, what mitigates today, what production needs.

Security hardening

docs/security/SECURITY_REVIEW.md: four attack scenarios walked through against the code, with the fix or the accepted risk.

UAT

docs/uat/UAT_PLAN.md: twelve scripts, executed on development and again on production after v0.3.0.

AI wrote most of the code. The rules it worked under, and the checks on its output, are in the repository.

BUILDING THE PRODUCT

Tools	Claude Code (Opus 5, some Fable 5.1) in Cursor's terminal; Cursor to read every diff before a commit; Wispr Flow to dictate the long briefs; Claude in Chrome for the dashboard steps and persona run-throughs; Notion, Railway, GitHub through MCP and CLIs.
Standing context	Two instruction files: a global engineering standard and the checked-in CLAUDE.md (three personas, layering, every status change through workflow.py, AI never mutates state, same-transaction audit, error envelope, the per-story checklist). The brief's five context types, each with an example in AI_USAGE.md.
How output was checked	Full suites on every story; browser checks at 390, 1024 and 1280; subagent reviews with separate briefs (design critique, edge cases, bug hunts, a strict assessor); every finding reproduced before a fix; docs checked against code at each sprint close.
Where it was wrong	Invented enum values and a valid verdict on an expired certificate (pinned schema, date in the prompt); a scope-creeping story and a red landing band (rejected); a transition the state machine did not allow (became Return to review); a scratch file swept into a commit.

DEBRIEF MATERIAL, SECOND SESSION

Skills	brag (launch video workflow); the HyperFrames set: hyperframes-core, -animation, -keyframes, -cli, -audio, media-use, general-video, slideshow; demo-video (Playwright footage recipe); claude-video-generator (narrated walkthrough rules).
Pipeline	Playwright records the development environment at 1920 x 1080 with a drawn cursor; narration through Kokoro via hyperframes tts; music from the HeyGen catalogue with my own sign-in; a HyperFrames composition checked and rendered at 60 fps.
Guardrails	Plans reviewed by four persona subagents through fourteen revisions before any build; the code, not the design documents, as the source of truth; a leak sweep of every recording for emails and ids; footage on development, never production.
Diagrams in this deck	ChatGPT image generation from prompts written against the workflows and docs; each picture rated against the code before use; two rejected for invented components (a CDN, an Azure tenant, a proxying frontend).

Known gaps first, then what production would need.

- 01

Use case 3
The checklist model, the officer capture screen with draft save, per-item clarification and the operator's targeted response. The statuses and transitions exist. About 1.5 days.
- 03

Object storage and backups
S3-compatible bucket with signed URLs and a virus scan; a file backup and a restore drill; a retention policy for documents and traces.
- 05

AI assurance
A labelled set from officer overrides; promptfoo or PyRIT red-team beyond two injection cases; Project Moonshot for Singapore evidence; calibrated confidence; in-region tracing; a multilingual injection classifier.
- 07

Delivery
Staging with a copy of production data for migration rehearsal; two replicas; Semgrep and CodeQL; Trivy with an SBOM on the images.

- 02

A worker for the AI checks
Redis and a worker, or a Postgres SKIP LOCKED queue, so checks survive deploys and scale apart from the API. ADR-004 has one call site to change.
- 04

Session and edge
httpOnly cookie with CSRF protection instead of sessionStorage; CSP nonces; the rate windows in Redis or at the edge.
- 06

Admin epic, v0.4.0
Overview with AI health and quota counters, audit feed, read-only case view, user management. Role and routes reserved, screens not built.
- 08

Product
Email delivery, officer assignment, more licence types via a configurable form schema, a public verification page for certificates.



Every decision has a file. Every claim has a test.



Thank you. Questions on any file, any decision, any gap.

permitflow.space
github.com/SuhaasNv/permitflow